Vercel AI SDK v6 complete code reference

This reference compiles copy-pasteable TypeScript for Vercel AI SDK v6 (April 2026) across Core, UI, Agents, Tools/MCP, Providers, Middleware, Testing, Telemetry, and Cookbook recipes. Every code block is reproduced verbatim from the official `ai-sdk.dev` docs. **All Core generation functions (`generateText`, `streamText`, `embed`, `embedMany`) now accept plain string model IDs** routed via the AI Gateway (e.g., `"anthropic/claude-sonnet-4.5"`), or explicit provider instances. **Structured output in v6 has been unified under `generateText`/`streamText` with `Output.object/array/choice/json/text`** — the legacy `generateObject`/`streamObject` remain but are de-emphasized. **The ToolLoopAgent class replaces ad-hoc agent loops** with built-in `stopWhen`, `prepareStep`, and `onStepFinish` hooks, and pairs with `DirectChatTransport` for in-process UI integration. ai-sdk + 5

---

1. Providers, registries, and model aliases

Custom providers and model aliasing

```typescript
```

```typescript
import {
  gateway,
  customProvider,
  defaultSettingsMiddleware,
  wrapLanguageModel,
} from 'ai';

export const openai = customProvider({
  languageModels: {
    'gpt-5.1': wrapLanguageModel({
      model: gateway('openai/gpt-5.1'),
      middleware: defaultSettingsMiddleware({
        settings: {
          providerOptions: { openai: { reasoningEffort: 'high' } },
        },
      }),
    }),
    'gpt-5.1-high-reasoning': wrapLanguageModel({
      model: gateway('openai/gpt-5.1'),
      middleware: defaultSettingsMiddleware({
        settings: {
          providerOptions: { openai: { reasoningEffort: 'high' } },
        },
      }),
    }),
  },
  fallbackProvider: gateway,
});
```

Provider registry

```typescript
```

```typescript
// registry.ts
import { anthropic } from '@ai-sdk/anthropic';
import { openai } from '@ai-sdk/openai';
import { createProviderRegistry, gateway } from 'ai';

export const registry = createProviderRegistry({
  gateway,
  anthropic,
  openai,
});

export const customSeparatorRegistry = createProviderRegistry(
  { gateway, anthropic, openai },
  { separator: ' > ' },
);
```

Accessors: `registry.languageModel('openai:gpt-5.1')`, `registry.embeddingModel('openai:text-embedding-3-small')`, `registry.imageModel('openai:dall-e-3')`. AI SDK ai-sdk

Combined registry + customProvider with aliases

```typescript
```

```
import { anthropic, AnthropicLanguageModelOptions } from '@ai-sdk/anthropic';
import { createOpenAICompatible } from '@ai-sdk/openai-compatible';
import { xai } from '@ai-sdk/xai';
import { groq } from '@ai-sdk/groq';
import {
  createProviderRegistry,
  customProvider,
  defaultSettingsMiddleware,
  gateway,
  wrapLanguageModel,
} from 'ai';

export const registry = createProviderRegistry(
  {
    gateway,
    xai,
    custom: createOpenAICompatible({
      name: 'provider-name',
      apiKey: process.env.CUSTOM_API_KEY,
      baseURL: 'https://api.custom.com/v1',
    }),
    anthropic: customProvider({
      languageModels: {
        fast: anthropic('claude-haiku-4-5'),
        writing: anthropic('claude-sonnet-4-5'),
        reasoning: wrapLanguageModel({
          model: anthropic('claude-sonnet-4-5'),
          middleware: defaultSettingsMiddleware({
            settings: {
              maxOutputTokens: 100000,
              providerOptions: {
                anthropic: {
                  thinking: { type: 'enabled', budgetTokens: 32000 },
                } satisfies AnthropicLanguageModelOptions,
              },
            },
          }),
        }),
      },
      fallbackProvider: anthropic,
    }),
    groq: customProvider({
      languageModels: {
```

```
        'gemma2-9b-it': groq('gemma2-9b-it'),
        'qwen-qwq-32b': groq('qwen-qwq-32b'),
      },
    }),
  },
  { separator: ' > ' },
);

const model = registry.languageModel('anthropic > reasoning');
```

Gateway string syntax and global provider

```typescript
import { streamText } from 'ai';

const result = await streamText({
  model: "anthropic/claude-sonnet-4.5",
  prompt: 'Invent a new holiday and describe its traditions.',
});

// Override the global provider at startup:
// globalThis.AI_SDK_DEFAULT_PROVIDER = openai;
```

OpenAI-compatible custom provider

```typescript
import { createOpenAICompatible } from '@ai-sdk/openai-compatible';

const custom = createOpenAICompatible({
  name: 'provider-name',
  apiKey: process.env.CUSTOM_API_KEY,
  baseURL: 'https://api.custom.com/v1',
  headers: { /* optional */ },
  queryParams: { /* optional */ },
  // fetch: customFetch,
});
```

2. Provider quick reference

OpenAI (`@ai-sdk/openai`)

```typescript
import { createOpenAI, openai } from '@ai-sdk/openai';
import { generateText } from 'ai';

const client = createOpenAI({
  apiKey: process.env.OPENAI_API_KEY,
  baseURL: 'https://api.openai.com/v1',
  organization: 'org_...',
  project: 'proj_...',
  headers: { 'header-name': 'header-value' },
});

const { text } = await generateText({
  model: openai('gpt-5'),
  prompt: 'Write a vegetarian lasagna recipe for 4 people.',
});
```

Provider-executed tools (Responses API):

```typescript
const { text, sources } = await generateText({
  model: openai('gpt-5'),
  prompt: 'What happened in San Francisco last week?',
  tools: {
    web_search: openai.tools.webSearch({ searchContextSize: 'low' }),
    code_interpreter: openai.tools.codeInterpreter(),
    file_search: openai.tools.fileSearch({
      vectorStoreIds: ['vs_1234'],
      maxNumResults: 5,
    }),
  },
});
```

Key providerOptions: `reasoningEffort`, `reasoningSummary`, `textVerbosity`, `parallelToolCalls`, `store`, `serviceTier`, `maxToolCalls`, `metadata`, `previousResponseId`, `user`. ai-sdk

Model IDs: `gpt-5`, `gpt-5-mini`, `gpt-5-nano`, `gpt-5.1`, `gpt-5.2`, `gpt-4.1`, `gpt-4o`, `o1`, `o3`, `o3-mini`, `o4-mini`. Embeddings: `text-embedding-3-small/large`. Images: `gpt-image-1`, `dall-e-3`.

Anthropic (`@ai-sdk/anthropic`)

```typescript
import { createAnthropic, anthropic } from '@ai-sdk/anthropic';

const client = createAnthropic({
  apiKey: process.env.ANTHROPIC_API_KEY,
  baseURL: 'https://api.anthropic.com/v1',
  headers: { 'anthropic-beta': 'computer-use-2025-01-24' },
});
```

Provider-executed tools:

```typescript
```

```
import { anthropic } from '@ai-sdk/anthropic';
import { generateText } from 'ai';
import { execSync } from 'node:child_process';

const computerTool = anthropic.tools.computer_20250124({
  displayWidthPx: 1024,
  displayHeightPx: 768,
  displayNumber: 1,
  execute: async ({ action, coordinate, text }) => {
    return { type: 'image', data: '<base64-png>' };
  },
  experimental_toToolResultContent: (result) => [
    { type: 'image', data: result.data, mediaType: 'image/png' },
  ],
});

const bashTool = anthropic.tools.bash_20250124({
  execute: async ({ command }) => execSync(command).toString(),
});

const textEditorTool = anthropic.tools.textEditor_20250124({
  execute: async ({ command, path, file_text, insert_line, new_str, old_str, view_ra
});

const webSearchTool = anthropic.tools.webSearch_20250305({
  maxUses: 5,
  allowedDomains: ['techcrunch.com', 'wired.com'],
  userLocation: {
    type: 'approximate',
    country: 'US', region: 'California', city: 'San Francisco',
    timezone: 'America/Los_Angeles',
  },
});

const response = await generateText({
  model: anthropic('claude-sonnet-4-5-20250929'),
  prompt: 'Take a screenshot, then search for the top story.',
  tools: {
    computer: computerTool,
    bash: bashTool,
    str_replace_editor: textEditorTool,
    web_search: webSearchTool,
```

```
  },
});
```

Key providerOptions:

```typescript
providerOptions: {
  anthropic: {
    thinking: { type: 'enabled', budgetTokens: 12000 },
    effort: 'high',
    speed: 'standard',
    cacheControl: { type: 'ephemeral' },
    sendReasoning: true,
    disableParallelToolUse: false,
    toolStreaming: true,
  } satisfies AnthropicLanguageModelOptions,
}
```

Model IDs: `claude-opus-4-5/6/7`, `claude-sonnet-4-5-20250929`, `claude-sonnet-4-6`, `claude-3-7-sonnet-20250219`, `claude-3-5-sonnet-latest`, `claude-3-5-haiku-latest`.

Google Generative AI (`@ai-sdk/google`)

```typescript
import { createGoogleGenerativeAI, google } from '@ai-sdk/google';

const client = createGoogleGenerativeAI({
  apiKey: process.env.GOOGLE_GENERATIVE_AI_API_KEY,
  baseURL: 'https://generativelanguage.googleapis.com/v1beta',
});

const { text, sources, providerMetadata } = await generateText({
  model: google('gemini-2.5-flash'),
  prompt: 'Summarize https://example.com/article.',
  tools: {
    google_search: google.tools.googleSearch({}),
    url_context: google.tools.urlContext({}),
    code_execution: google.tools.codeExecution({}),
  },
});
```

Key providerOptions:

```typescript
providerOptions: {
  google: {
    thinkingConfig: {
      thinkingBudget: 2024,
      includeThoughts: true,
      thinkingLevel: 'medium',
    },
    responseModalities: ['TEXT'],
    safetySettings: [
      { category: 'HARM_CATEGORY_HARASSMENT', threshold: 'BLOCK_MEDIUM_AND_ABOVE' },
    ],
    structuredOutputs: true,
  } satisfies GoogleGenerativeAIProviderOptions,
}
```

Model IDs: `gemini-3-pro-preview`, `gemini-2.5-pro/flash/flash-lite`, `gemini-2.0-flash`, `gemini-1.5-pro/flash`. AI SDK

Groq and xAI

```typescript
import { createGroq, groq } from '@ai-sdk/groq';
const g = createGroq({ apiKey: process.env.GROQ_API_KEY });

await generateText({
  model: groq('openai/gpt-oss-120b'),
  prompt: 'What are the latest developments in AI?',
  tools: { browser_search: groq.tools.browserSearch({}) },
  toolChoice: 'required',
});
```

```typescript
```

```
import { createXai, xai } from '@ai-sdk/xai';

const { fullStream } = streamText({
  model: xai.responses('grok-4.20-reasoning'),
  prompt: 'What are the latest updates from xAI?',
  tools: {
    web_search: xai.tools.webSearch(),
    x_search: xai.tools.xSearch(),
    code_execution: xai.tools.codeExecution(),
  },
});
```

---

3. Generating text and structured data

Basic generateText and streamText

```typescript
```

```
import { generateText, streamText } from 'ai';

const { text } = await generateText({
  model: "anthropic/claude-sonnet-4.5",
  system: 'You write simple, clear, and concise content.',
  prompt: `Summarize: ${article}`,
});

const result = streamText({
  model: "anthropic/claude-sonnet-4.5",
  prompt: 'Invent a new holiday.',
  onChunk({ chunk }) {
    if (chunk.type === 'text') console.log(chunk.text);
  },
  onError({ error }) { console.error(error); },
  onFinish({ text, finishReason, usage, response, steps, totalUsage }) {
    // save chat history, record usage
  },
});

for await (const textPart of result.textStream) {
  console.log(textPart);
}
```

Full stream with all part types

typescript

```
for await (const part of result.fullStream) {
  switch (part.type) {
    case 'start': case 'start-step': break;
    case 'text-start': case 'text-delta': case 'text-end': break;
    case 'reasoning-start': case 'reasoning-delta': case 'reasoning-end': break;
    case 'source': case 'file': break;
    case 'tool-call': case 'tool-input-start': case 'tool-input-delta':
    case 'tool-input-end': case 'tool-result': case 'tool-error': break;
    case 'finish-step': case 'finish': case error': case 'raw': break;
  }
}
```

ai-sdk

Structured output with `Output.*`

typescript

```
import { generateText, streamText, Output } from 'ai';
import { z } from 'zod';

// Object with Zod schema
const { output } = await generateText({
  model: "anthropic/claude-sonnet-4.5",
  output: Output.object({
    name: 'Recipe',
    description: 'A recipe for a dish.',
    schema: z.object({
      name: z.string(),
      ingredients: z.array(z.object({ name: z.string(), amount: z.string() })),
      steps: z.array(z.string()),
    }),
  }),
  prompt: 'Generate a lasagna recipe.',
});

// Streaming partial object
const { partialOutputStream } = streamText({
  model: "anthropic/claude-sonnet-4.5",
  output: Output.object({ schema }),
  prompt: 'Generate a lasagna recipe.',
});
for await (const partialObject of partialOutputStream) {
  console.log(partialObject);
}

// Array with elementStream (each element validated complete)
const { elementStream } = streamText({
  output: Output.array({
    element: z.object({ name: z.string(), class: z.string(), description: z.string()
  }),
  prompt: 'Generate 3 hero descriptions.',
});
for await (const hero of elementStream) { console.log(hero); }

// Enum-like classification
const { output: weather } = await generateText({
  output: Output.choice({ options: ['sunny', 'rainy', 'snowy'] }),
  prompt: 'Is the weather sunny, rainy, or snowy?',
});
```

```
// No-schema JSON, or plain text
Output.json();
Output.text();
```

Settings and timeouts

Common settings: `maxOutputTokens`, `temperature`, `topP`, `topK`, `presencePenalty`, `frequencyPenalty`, `stopSequences`, `seed`, `maxRetries`, `abortSignal`, `headers`. **Temperature is no longer defaulted to 0 in v5+.** ai-sdk + 2

```typescript
const result = await generateText({
  model: "anthropic/claude-sonnet-4.5",
  prompt: 'Invent a new holiday.',
  abortSignal: AbortSignal.timeout(5000),
  timeout: { totalMs: 60000, stepMs: 10000 },
  // streamText-only: timeout: { chunkMs: 5000 }
});
```

Prompts: text, messages, multi-modal

```typescript
```

```
// Image URL, buffer, or base64
messages: [{
  role: 'user',
  content: [
    { type: 'text', text: 'Describe the image.' },
    { type: 'image', image: fs.readFileSync('./cat.png') },
  ],
}]

// PDF or audio file
messages: [{
  role: 'user',
  content: [
    { type: 'text', text: 'What is the file about?' },
    {
      type: 'file',
      mediaType: 'application/pdf',
      data: fs.readFileSync('./example.pdf'),
      filename: 'example.pdf',
    },
  ],
}]

// Tool round-trip
messages: [
  { role: 'assistant', content: [{ type: 'tool-call', toolCallId: '12345', toolName:
  { role: 'tool', content: [{
    type: 'tool-result', toolCallId: '12345', toolName: 'get-nutrition',
    output: { type: 'json', value: { calories: 369, fat: 31, protein: 22 } },
  }] },
]
```

Error handling, abort, NoObjectGenerated

typescript

```typescript
import { APICallError, NoObjectGeneratedError, streamText, generateText } from 'ai';

try {
  await generateText({ model, prompt });
} catch (error) {
  if (APICallError.isInstance(error)) {
    // error.url, statusCode, responseHeaders, responseBody, cause, isRetryable
  }
  if (NoObjectGeneratedError.isInstance(error)) {
    // error.cause, error.text, error.response, error.usage, error.finishReason
  }
}

const { textStream } = streamText({
  model, prompt,
  onAbort: ({ steps }) => console.log('Aborted after', steps.length, 'steps'),
  onFinish: ({ steps, totalUsage }) => console.log('Completed normally'),
});
```

---

4. Tools, MCP, and agents

Tool definition with all hooks

```typescript
```

```
import { tool, streamText } from 'ai';
import { z } from 'zod';

const weatherTool = tool({
  description: 'Get the weather in a location',
  inputSchema: z.object({ location: z.string() }),
  strict: true,
  inputExamples: [{ input: { location: 'San Francisco' } }],
  needsApproval: async ({ amount }) => amount > 1000,
  onInputStart: () => console.log('Tool call starting'),
  onInputDelta: ({ inputTextDelta }) => console.log(inputTextDelta),
  onInputAvailable: ({ input }) => console.log('Complete:', input),
  execute: async ({ location }, { toolCallId, messages, abortSignal, experimental_co
    return { location, temperature: 72 };
  },
});
```

Async-iterable `execute` for preliminary yields:

```typescript
async *execute({ location }) {
  yield { status: 'loading' as const, text: `Getting weather for ${location}` };
  await new Promise(r => setTimeout(r, 3000));
  yield { status: 'success' as const, temperature: 72 };
}
```

Multi-step with `stopWhen` and `prepareStep`

```typescript
```

```typescript
import { generateText, stepCountIs, hasToolCall } from 'ai';

const { text, steps } = await generateText({
  model: "anthropic/claude-sonnet-4.5",
  tools: { weather: weatherTool },
  stopWhen: [stepCountIs(5), hasToolCall('someTool')],
  prepareStep: async ({ model, stepNumber, steps, messages }) => {
    if (stepNumber === 0) {
      return {
        model: modelForThisStep,
        toolChoice: { type: 'tool', toolName: 'tool1' },
        activeTools: ['tool1'],
      };
    }
    if (messages.length > 20) {
      return { messages: messages.slice(-10) };
    }
  },
  onStepFinish({ stepNumber, finishReason, usage, toolCalls, toolResults }) {
    console.log(`Step ${stepNumber}: ${finishReason}`);
  },
  prompt: 'What is the weather in SF?',
});
```

Approval flow (human-in-the-loop)

typescript

```
import { type ModelMessage, type ToolApprovalResponse } from 'ai';

const messages: ModelMessage[] = [{ role: 'user', content: 'Remove the most recent f
const result = await generateText({ model, tools: { runCommand }, messages });
messages.push(...result.response.messages);

const approvals: ToolApprovalResponse[] = [];
for (const part of result.content) {
  if (part.type === 'tool-approval-request') {
    approvals.push({
      type: 'tool-approval-response',
      approvalId: part.approvalId,
      approved: true,
      reason: 'User confirmed',
    });
  }
}
messages.push({ role: 'tool', content: approvals });
```

`experimental_repairToolCall`

typescript

```
experimental_repairToolCall: async ({ toolCall, tools, inputSchema, error }) => {
  if (NoSuchToolError.isInstance(error)) return null;
  const tool = tools[toolCall.toolName as keyof typeof tools];
  const { output: repairedArgs } = await generateText({
    model,
    output: Output.object({ schema: tool.inputSchema }),
    prompt: `The model tried to call "${toolCall.toolName}" with: ${JSON.stringify(t
  });
  return { ...toolCall, input: JSON.stringify(repairedArgs) };
},
```

MCP client with all transports

typescript

```
import { createMCPClient } from '@ai-sdk/mcp';
import { Experimental_StdioMCPTransport } from '@ai-sdk/mcp/mcp-stdio';
import { StreamableHTTPClientTransport } from '@modelcontextprotocol/sdk/client/stre

// HTTP (recommended for production)
const httpClient = await createMCPClient({
  transport: {
    type: 'http',
    url: 'https://your-server.com/mcp',
    headers: { Authorization: 'Bearer my-api-key' },
    authProvider: myOAuthClientProvider,
    redirect: 'error',
  },
});

// SSE
const sseClient = await createMCPClient({
  transport: { type: 'sse', url: 'https://my-server.com/sse' },
});

// Streamable HTTP with session
const streamableClient = await createMCPClient({
  transport: new StreamableHTTPClientTransport(
    new URL('https://your-server.com/mcp'),
    { sessionId: 'session_123' },
  ),
});

// Stdio (local dev only)
const stdioClient = await createMCPClient({
  transport: new Experimental_StdioMCPTransport({
    command: 'node',
    args: ['src/stdio/dist/server.js'],
  }),
});

// Tool discovery with typed schemas
const tools = await httpClient.tools({
  schemas: {
    'get-weather': {
      inputSchema: z.object({ location: z.string() }),
      outputSchema: z.object({ temperature: z.number(), conditions: z.string() }),
    },
```

```
  },
});

// Always close
const result = streamText({
  model, tools, prompt: 'What is the weather?',
  onFinish: async () => await httpClient.close(),
});
```

ToolLoopAgent

```typescript
```

```
import { ToolLoopAgent, Output, stepCountIs, isLoopFinished, InferAgentUIMessage } f
import { z } from 'zod';

const agent = new ToolLoopAgent({
  model: "anthropic/claude-sonnet-4.5",
  instructions: 'You are a senior software engineer.',
  tools: { weather: weatherTool, analyze: analyzeTool },
  toolChoice: 'auto',
  stopWhen: [stepCountIs(50)],
  output: Output.object({
    schema: z.object({
      sentiment: z.enum(['positive', 'neutral', 'negative']),
      summary: z.string(),
    }),
  }),
  prepareStep: async ({ stepNumber, messages }) => {
    if (stepNumber > 2 && messages.length > 10) {
      return { model: "anthropic/claude-sonnet-4.5" };
    }
    return {};
  },
  onStepFinish: async ({ stepNumber, usage }) => {
    console.log(`Step ${stepNumber}:`, usage.totalTokens);
  },
});

const { text, output, steps } = await agent.generate({ prompt: 'Analyze feedback' })
const stream = await agent.stream({ prompt: 'Tell me a story' });

export type MyAgentUIMessage = InferAgentUIMessage<typeof agent>;
```

Agent API route via `createAgentUIStreamResponse`:

```typescript
import { createAgentUIStreamResponse } from 'ai';

export async function POST(request: Request) {
  const { messages } = await request.json();
  return createAgentUIStreamResponse({ agent, uiMessages: messages });
}
```

Workflow patterns

Sequential chains, routing by classification, parallel fan-out, orchestrator-worker, and evaluator-optimizer loops follow the same primitive: chain `generateText` calls with `Output.object` schemas between each step. A typical evaluator-optimizer loop iterates up to N times, re-generating when evaluation falls below threshold — break when `qualityScore >= 8 && preservesTone && preservesNuance`. ai-sdk Laravel

---

5. Embeddings, media, middleware, testing

Embeddings and similarity

typescript

```typescript
import { embed, embedMany, cosineSimilarity, wrapEmbeddingModel, defaultEmbeddingSet

const { embedding } = await embed({
  model: 'openai/text-embedding-3-small',
  value: 'sunny day at the beach',
  providerOptions: { openai: { dimensions: 512 } },
});

const { embeddings } = await embedMany({
  maxParallelCalls: 2,
  model: 'openai/text-embedding-3-small',
  values: ['sunny day', 'rainy afternoon', 'snowy night'],
});

const similarity = cosineSimilarity(embeddings[0], embeddings[1]);

const modelWithDefaults = wrapEmbeddingModel({
  model: gateway.embeddingModel('google/gemini-embedding-001'),
  middleware: defaultEmbeddingSettingsMiddleware({
    settings: {
      providerOptions: {
        google: { outputDimensionality: 256, taskType: 'CLASSIFICATION' },
      },
    },
  }),
});
```

Image, transcription, speech

```typescript
import { generateImage, experimental_transcribe as transcribe, experimental_generate
import { openai } from '@ai-sdk/openai';

const { image, images } = await generateImage({
  model: "openai/gpt-image-1",
  prompt: 'Santa Claus driving a Cadillac',
  size: '1024x1024',
  aspectRatio: '16:9',
  n: 4,
  maxImagesPerCall: 5,
  seed: 1234567890,
  providerOptions: { openai: { style: 'vivid', quality: 'hd' } },
});

const transcript = await transcribe({
  model: openai.transcription('whisper-1'),
  audio: await readFile('audio.mp3'),
  providerOptions: { openai: { timestampGranularities: ['word'] } },
});

const audio = await generateSpeech({
  model: openai.speech('tts-1'),
  text: 'Hello, world!',
  voice: 'alloy',
  language: 'en',
});
```

Language model middleware

Built-ins: `extractReasoningMiddleware({ tagName: 'think' })`, `extractJsonMiddleware()`, `simulateStreamingMiddleware()`, `defaultSettingsMiddleware({ settings })`, `addToolInputExamplesMiddleware({ prefix, format, remove })`.

Custom logging middleware:

```typescript
```

```
import type { LanguageModelV3Middleware, LanguageModelV3StreamPart } from '@ai-sdk/p

export const loggingMiddleware: LanguageModelV3Middleware = {
  wrapGenerate: async ({ doGenerate, params }) => {
    console.log('params:', JSON.stringify(params, null, 2));
    const result = await doGenerate();
    console.log('generated text:', result.text);
    return result;
  },
  wrapStream: async ({ doStream, params }) => {
    const { stream, ...rest } = await doStream();
    let generatedText = '';
    const textBlocks = new Map<string, string>();
    const transformStream = new TransformStream<LanguageModelV3StreamPart, LanguageM
      transform(chunk, controller) {
        switch (chunk.type) {
          case 'text-start': textBlocks.set(chunk.id, ''); break;
          case 'text-delta':
            textBlocks.set(chunk.id, (textBlocks.get(chunk.id) || '') + chunk.delta)
            generatedText += chunk.delta;
            break;
          case 'text-end': console.log('block done:', textBlocks.get(chunk.id)); bre
        }
        controller.enqueue(chunk);
      },
      flush() { console.log('generated:', generatedText); },
    });
    return { stream: stream.pipeThrough(transformStream), ...rest };
  },
};
```

RAG-injection middleware uses `transformParams` to prepend retrieved context to the last user message. Guardrails middleware wraps `doGenerate()` and regex-redacts sensitive terms. Apply via `wrapLanguageModel({ model, middleware: [first, second] })` — outer-first order.

Testing utilities

typescript

```typescript
import { streamText, Output, simulateReadableStream } from 'ai';
import { MockLanguageModelV3 } from 'ai/test';
import { z } from 'zod';

const result = streamText({
  model: new MockLanguageModelV3({
    doStream: async () => ({
      stream: simulateReadableStream({
        chunks: [
          { type: 'text-start', id: 'text-1' },
          { type: 'text-delta', id: 'text-1', delta: '{ ' },
          { type: 'text-delta', id: 'text-1', delta: '"content": "Hello!"' },
          { type: 'text-delta', id: 'text-1', delta: ' }' },
          { type: 'text-end', id: 'text-1' },
          { type: 'finish', finishReason: { unified: 'stop', raw: undefined }, usage
        ],
      }),
    }),
  }),
  output: Output.object({ schema: z.object({ content: z.string() }) }),
  prompt: 'Hello, test!',
});
```

Telemetry

typescript

```typescript
const result = await generateText({
  model: "anthropic/claude-sonnet-4.5",
  prompt: 'Write a story.',
  experimental_telemetry: {
    isEnabled: true,
    functionId: 'my-function',
    metadata: { userId: '123' },
    recordInputs: true,
    recordOutputs: true,
    tracer: tracerProvider.getTracer('ai'),
    integrations: [devToolsIntegration(), customLogger()],
  },
});
```

Integrations implement `TelemetryIntegration` with lifecycle methods `onStart`, `onStepStart`, `onToolCallStart`, `onToolCallFinish`, `onStepFinish`, `onFinish`. Wrap class-based integrations with `bindTelemetryIntegration(instance)`.

---

6. UI layer: useChat, transport, tools, persistence

Minimal chatbot (client + server)

```tsx
```

```
// app/page.tsx
'use client';
import { useChat } from '@ai-sdk/react';
import { DefaultChatTransport } from 'ai';
import { useState } from 'react';

export default function Page() {
  const { messages, sendMessage, status, stop, error, regenerate } = useChat({
    transport: new DefaultChatTransport({ api: '/api/chat' }),
  });
  const [input, setInput] = useState('');

  return (
    <>
      {messages.map(m => (
        <div key={m.id}>
          {m.role}:
          {m.parts.map((part, i) =>
            part.type === 'text' ? <span key={i}>{part.text}</span> : null,
          )}
        </div>
      ))}
      {(status === 'submitted' || status === 'streaming') && (
        <button onClick={() => stop()}>Stop</button>
      )}
      {error && <button onClick={() => regenerate()}>Retry</button>}
      <form onSubmit={e => {
        e.preventDefault();
        if (input.trim()) { sendMessage({ text: input }); setInput(''); }
      }}>
        <input value={input} onChange={e => setInput(e.target.value)} disabled={stat
      </form>
    </>
  );
}
```

ts

```
// app/api/chat/route.ts
import { convertToModelMessages, streamText, UIMessage } from 'ai';

export const maxDuration = 30;

export async function POST(req: Request) {
  const { messages }: { messages: UIMessage[] } = await req.json();
  const result = streamText({
    model: 'anthropic/claude-sonnet-4.5',
    messages: await convertToModelMessages(messages),
  });
  return result.toUIMessageStreamResponse({
    sendReasoning: true,
    sendSources: true,
    onError: error => error instanceof Error ? error.message : 'unknown error',
  });
}
```

`status` values: `submitted` | `streaming` | `ready` | `error`. Messages are rendered via `message.parts[]` — never `content`.

Transport configuration

```tsx
```

```ts
import { DefaultChatTransport, DirectChatTransport } from 'ai';

new DefaultChatTransport({
  api: '/api/chat',
  headers: () => ({ Authorization: `Bearer ${getAuthToken()}` }),
  body: () => ({ sessionId: getCurrentSessionId() }),
  credentials: 'include',
  prepareSendMessagesRequest: ({ id, messages, trigger, messageId }) => {
    if (trigger === 'submit-user-message') {
      return { body: { id, message: messages[messages.length - 1], messageId } };
    }
    if (trigger === 'regenerate-assistant-message') {
      return { body: { trigger, id, messageId } };
    }
    throw new Error(`Unsupported trigger: ${trigger}`);
  },
  prepareReconnectToStreamRequest: ({ id }) => ({
    api: `/api/chat/${id}/stream`,
    credentials: 'include',
    headers: { Authorization: 'Bearer token' },
  }),
});

// In-process (no HTTP)
new DirectChatTransport({ agent, sendReasoning: true, sendSources: true });
```

Per-request override: `sendMessage({ text }, { headers, body, metadata })`.

Tool usage in the UI

Server route with three tool types:

```ts
```

```
tools: {
  getWeatherInformation: {
    description: 'show the weather',
    inputSchema: z.object({ city: z.string() }),
    execute: async ({ city }) => 'sunny',
  },
  askForConfirmation: {
    description: 'Ask user for confirmation',
    inputSchema: z.object({ message: z.string() }),
  },
  getLocation: {
    description: 'Get user location',
    inputSchema: z.object({}),
  },
},
```

Client with `onToolCall`, `addToolOutput`, and state-based rendering:

```tsx
```

```
import { useChat } from '@ai-sdk/react';
import { DefaultChatTransport, lastAssistantMessageIsCompleteWithToolCalls } from 'a

const { messages, sendMessage, addToolOutput, addToolApprovalResponse } = useChat({
  transport: new DefaultChatTransport({ api: '/api/chat' }),
  sendAutomaticallyWhen: lastAssistantMessageIsCompleteWithToolCalls,
  async onToolCall({ toolCall }) {
    if (toolCall.dynamic) return;
    if (toolCall.toolName === 'getLocation') {
      addToolOutput({
        tool: 'getLocation',
        toolCallId: toolCall.toolCallId,
        output: 'San Francisco',
      });
    }
  },
});

message.parts.map(part => {
  switch (part.type) {
    case 'tool-askForConfirmation':
      if (part.state === 'input-available') {
        return (
          <div>
            {part.input.message}
            <button onClick={() => addToolOutput({ tool: 'askForConfirmation', toolC
          </div>
        );
      }
      if (part.state === 'output-available') return <div>{part.output}</div>;
      if (part.state === 'output-error') return <div>Error: {part.errorText}</div>;
      break;
    case 'tool-getWeatherInformation':
      if (part.state === 'output-available') return <div>Weather in {part.input.city
      break;
    case 'dynamic-tool':
      return <div>Tool: {part.toolName}</div>;
    case 'step-start':
      return <hr />;
  }
});
```

Approval flow: server tool with `needsApproval: true`, client renders `part.state === 'approval-requested'` and calls `addToolApprovalResponse({ id: part.approval.id, approved: true })`. Auto-continue with `sendAutomaticallyWhen: lastAssistantMessageIsCompleteWithApprovalResponses`.

Message persistence

```ts
// util/chat-store.ts
import { generateId, UIMessage } from 'ai';
import { readFile, writeFile } from 'fs/promises';

export async function createChat(): Promise<string> {
  const id = generateId();
  await writeFile(getChatFile(id), '[]');
  return id;
}

export async function loadChat(id: string): Promise<UIMessage[]> {
  return JSON.parse(await readFile(getChatFile(id), 'utf8'));
}

export async function saveChat({ chatId, messages }: { chatId: string; messages: UIM
  await writeFile(getChatFile(chatId), JSON.stringify(messages, null, 2));
}
```

Server route persisting via `onFinish` with `validateUIMessages`:

```ts
```

```
import { convertToModelMessages, streamText, validateUIMessages, TypeValidationError

export async function POST(req: Request) {
  const { message, id } = await req.json();
  const previousMessages = await loadChat(id);
  let validatedMessages;
  try {
    validatedMessages = await validateUIMessages({
      messages: [...previousMessages, message],
      tools, metadataSchema, dataPartsSchema,
    });
  } catch (error) {
    if (error instanceof TypeValidationError) validatedMessages = [];
    else throw error;
  }

  const result = streamText({
    model: 'openai/gpt-5-mini',
    messages: convertToModelMessages(validatedMessages),
  });

  result.consumeStream(); // ensures onFinish fires on client disconnect

  return result.toUIMessageStreamResponse({
    originalMessages: validatedMessages,
    generateMessageId: createIdGenerator({ prefix: 'msg', size: 16 }),
    onFinish: ({ messages }) => saveChat({ chatId: id, messages }),
  });
}
```

Custom data streams

ts

```ts
// ai/types.ts
import { UIMessage } from 'ai';

export type MyUIMessage = UIMessage<
  never,
  {
    weather: { city: string; weather?: string; status: 'loading' | 'success' };
    notification: { message: string; level: 'info' | 'warning' | 'error' };
  }
>;
```

ts

```
// Server
import { createUIMessageStream, createUIMessageStreamResponse, streamText, generateI

const stream = createUIMessageStream<MyUIMessage>({
  execute: ({ writer }) => {
    writer.write({
      type: 'data-notification',
      data: { message: 'Processing...', level: 'info' },
      transient: true, // only accessible via onData
    });
    writer.write({
      type: 'source',
      value: { type: 'source', sourceType: 'url', id: 'source-1', url: 'https://weat
    });
    writer.write({
      type: 'data-weather',
      id: 'weather-1', // same id reconciles updates
      data: { city: 'SF', status: 'loading' },
    });
    writer.write({ type: 'start', messageId: generateId() });

    const result = streamText({ model, messages, onFinish() {
      writer.write({
        type: 'data-weather',
        id: 'weather-1',
        data: { city: 'SF', weather: 'sunny', status: 'success' },
      });
    }});

    writer.merge(result.toUIMessageStream({ sendStart: false }));
  },
  originalMessages: messages,
  onFinish: ({ responseMessage }) => { /* persist */ },
});

return createUIMessageStreamResponse({ stream });
```

tsx

```
// Client onData handler
const { messages } = useChat<MyUIMessage>({
  api: '/api/chat',
  onData: dataPart => {
    if (dataPart.type === 'data-weather') console.log('Weather update:', dataPart.da
    if (dataPart.type === 'data-notification') showToast(dataPart.data.message);
  },
});
```

Message metadata

```ts
// Schema
import { UIMessage } from 'ai';
import { z } from 'zod';

export const messageMetadataSchema = z.object({
  createdAt: z.number().optional(),
  model: z.string().optional(),
  totalTokens: z.number().optional(),
});
export type MyUIMessage = UIMessage<z.infer<typeof messageMetadataSchema>>;

// Server
return result.toUIMessageStreamResponse({
  originalMessages: messages,
  messageMetadata: ({ part }) => {
    if (part.type === 'start') return { createdAt: Date.now(), model: 'gpt-5.1' };
    if (part.type === 'finish') return { totalTokens: part.totalUsage.totalTokens };
  },
});

// Client access: message.metadata?.createdAt, message.metadata?.totalTokens
```

Reading UI message streams server-side

```ts
```

```
import { readUIMessageStream, streamText } from 'ai';

const result = streamText({ model, prompt: 'Write a story.' });

for await (const uiMessage of readUIMessageStream({
  stream: result.toUIMessageStream(),
})) {
  uiMessage.parts.forEach(part => {
    switch (part.type) {
      case 'text': console.log('Text:', part.text); break;
      case 'tool-call': console.log('Tool:', part.toolName, part.args); break;
      case 'tool-result': console.log('Result:', part.result); break;
    }
  });
}
```

Generative UI (typed tool components)

```tsx
```

```
// components/weather.tsx
export const Weather = ({ temperature, weather, location }: { temperature: number; w
  <div>
    <h2>Current Weather for {location}</h2>
    <p>Condition: {weather}</p>
    <p>Temperature: {temperature}°C</p>
  </div>
);

// ai/tools.ts
import { tool as createTool } from 'ai';
import { z } from 'zod';

export const weatherTool = createTool({
  description: 'Display the weather for a location',
  inputSchema: z.object({ location: z.string() }),
  execute: async ({ location }) => ({ weather: 'Sunny', temperature: 75, location })
});

// app/page.tsx — rendering by tool state
if (part.type === 'tool-displayWeather') {
  switch (part.state) {
    case 'input-available': return <div>Loading weather...</div>;
    case 'output-available': return <Weather {...part.output} />;
    case 'output-error': return <div>Error: {part.errorText}</div>;
  }
}
```

Resumable streams

tsx

```
// Client
const { messages, sendMessage } = useChat({
  id: chatData.id,
  messages: chatData.messages,
  resume: true,
  transport: new DefaultChatTransport({
    prepareSendMessagesRequest: ({ id, messages }) => ({
      body: { id, message: messages[messages.length - 1] },
    }),
  }),
});
```

ts

```
// POST handler with consumeSseStream
import { createResumableStreamContext } from 'resumable-stream';
import { after } from 'next/server';

return result.toUIMessageStreamResponse({
  originalMessages: messages,
  generateMessageId: generateId,
  onFinish: ({ messages }) => saveChat({ id, messages, activeStreamId: null }),
  async consumeSseStream({ stream }) {
    const streamId = generateId();
    const streamContext = createResumableStreamContext({ waitUntil: after });
    await streamContext.createNewResumableStream(streamId, () => stream);
    saveChat({ id, activeStreamId: streamId });
  },
});

// GET /api/chat/[id]/stream
import { UI_MESSAGE_STREAM_HEADERS } from 'ai';

export async function GET(_: Request, { params }: { params: Promise<{ id: string }>
  const { id } = await params;
  const chat = await readChat(id);
  if (chat.activeStreamId == null) return new Response(null, { status: 204 });
  const streamContext = createResumableStreamContext({ waitUntil: after });
  return new Response(
    await streamContext.resumeExistingStream(chat.activeStreamId),
    { headers: UI_MESSAGE_STREAM_HEADERS },
  );
}
```

useCompletion and useObject

```
tsx
```

```
// Completion
import { useCompletion } from '@ai-sdk/react';

const { completion, input, handleInputChange, handleSubmit, isLoading, stop, error }
  api: '/api/completion',
  headers: { Authorization: 'your_token' },
  body: { user_id: '123' },
  onFinish: (prompt, completion) => console.log(completion),
});
```

```tsx
// Object generation with partial streaming
import { experimental_useObject as useObject } from '@ai-sdk/react';
import { notificationSchema } from './api/notifications/schema';

const { object, submit, isLoading, stop, error, clear } = useObject({
  api: '/api/notifications',
  schema: notificationSchema,
  onFinish({ object, error }) {
    console.log('Completed:', object);
  },
});

// Render partial objects with optional chaining
{object?.notifications?.map((n, i) => (
  <div key={i}>
    <p>{n?.name}</p>
    <p>{n?.message}</p>
  </div>
))}
```

Server companion uses `streamText` with `Output.object` returning `result.toTextStreamResponse()`.

---

7. UI message stream protocol

The v6 UI Message Stream protocol (header `x-vercel-ai-ui-message-stream: v1`) defines these event types:

- `start` / `finish` — message boundaries with metadata and totalUsage

- `text-start` / `text-delta` / `text-end` — id-keyed text blocks
- `reasoning-start` / `reasoning-delta` / `reasoning-end` — id-keyed reasoning blocks
- `tool-input-start` / `tool-input-delta` / `tool-input-available` / `tool-output-available` / `tool-output-error` — tool lifecycle
- `source` — URL or document citations
- `file` — file parts with mediaType and url
- `data-${name}` — custom data parts, with optional `transient: true`
- `error` — error events
- `message-metadata` — metadata from the `messageMetadata` callback

On the client, `message.parts[]` exposes these as typed parts — tool parts appear as `tool-${toolName}` with a `state` field of `input-streaming` | `input-available` | `output-available` | `output-error` | `approval-requested`, and unknown tools surface as `dynamic-tool`.

---

8. Cookbook recipes

MCP tools (Node)

```ts
```

```
import { createMCPClient } from '@ai-sdk/mcp';
import { generateText, stepCountIs } from 'ai';
import { Experimental_StdioMCPTransport } from '@ai-sdk/mcp/mcp-stdio';

let clientOne, clientTwo, clientThree;
try {
  clientOne = await createMCPClient({
    transport: new Experimental_StdioMCPTransport({
      command: 'node',
      args: ['src/stdio/dist/server.js'],
    }),
  });
  clientTwo = await createMCPClient({
    transport: { type: 'http', url: 'http://localhost:3000/mcp' },
  });
  clientThree = await createMCPClient({
    transport: { type: 'sse', url: 'http://localhost:3000/sse' },
  });

  const tools = {
    ...(await clientOne.tools()),
    ...(await clientTwo.tools()),
    ...(await clientThree.tools()),
  };

  const response = await generateText({
    model: 'openai/gpt-4o',
    tools,
    stopWhen: stepCountIs(5),
    messages: [{ role: 'user', content: [{ type: 'text', text: 'Find products under
  });
} finally {
  await Promise.all([clientOne?.close(), clientTwo?.close(), clientThree?.close()]);
}
```

RAG chatbot setup (Next.js + Drizzle + pgvector)

```bash
```

```
git clone https://github.com/vercel/ai-sdk-rag-starter
cd ai-sdk-rag-starter
pnpm install
cp .env.example .env
pnpm db:migrate
pnpm add ai @ai-sdk/react
```

```
# .env
DATABASE_URL=your-postgres-connection-string
AI_GATEWAY_API_KEY=your-api-key
```

ts

```ts
// lib/db/schema/embeddings.ts
import { nanoid } from '@/lib/utils';
import { index, pgTable, text, varchar, vector } from 'drizzle-orm/pg-core';
import { resources } from './resources';

export const embeddings = pgTable(
  'embeddings',
  {
    id: varchar('id', { length: 191 }).primaryKey().$defaultFn(() => nanoid()),
    resourceId: varchar('resource_id', { length: 191 }).references(
      () => resources.id, { onDelete: 'cascade' },
    ),
    content: text('content').notNull(),
    embedding: vector('embedding', { dimensions: 1536 }).notNull(),
  },
  table => ({
    embeddingIndex: index('embeddingIndex').using('hnsw', table.embedding.op('vector
  }),
);
```

ts

```ts
// lib/ai/embedding.ts
import { embedMany } from 'ai';

const embeddingModel = 'openai/text-embedding-ada-002';

const generateChunks = (input: string): string[] =>
  input.trim().split('.').filter(i => i !== '');

export const generateEmbeddings = async (
  value: string,
): Promise<Array<{ embedding: number[]; content: string }>> => {
  const chunks = generateChunks(value);
  const { embeddings } = await embedMany({ model: embeddingModel, values: chunks });
  return embeddings.map((e, i) => ({ content: chunks[i], embedding: e }));
};
```

ts

```
// app/api/chat/route.ts
import { createResource } from '@/lib/actions/resources';
import { convertToModelMessages, streamText, tool, UIMessage } from 'ai';
import { z } from 'zod';

export const maxDuration = 30;

export async function POST(req: Request) {
  const { messages }: { messages: UIMessage[] } = await req.json();
  const result = streamText({
    model: 'openai/gpt-4o',
    system: `You are a helpful assistant. Check your knowledge base before answering
    Only respond using tool-call information.
    If no relevant information is found, respond: "Sorry, I don't know."`,
    messages: await convertToModelMessages(messages),
    tools: {
      addResource: tool({
        description: 'add a resource to your knowledge base',
        inputSchema: z.object({ content: z.string() }),
        execute: async ({ content }) => createResource({ content }),
      }),
    },
  });
  return result.toUIMessageStreamResponse();
}
```

Multi-modal chatbot (file upload with images + PDFs)

```tsx
```

```
async function convertFilesToDataURLs(files: FileList) {
  return Promise.all(
    Array.from(files).map(file => new Promise<{ type: 'file'; mediaType: string; url
      const reader = new FileReader();
      reader.onload = () => resolve({ type: 'file', mediaType: file.type, url: reade
      reader.onerror = reject;
      reader.readAsDataURL(file);
    })),
  );
}

// On submit:
const fileParts = files && files.length > 0 ? await convertFilesToDataURLs(files) :
sendMessage({
  role: 'user',
  parts: [{ type: 'text', text: input }, ...fileParts],
});

// Rendering:
if (part.type === 'file' && part.mediaType?.startsWith('image/')) {
  return <Image src={part.url} width={500} height={500} alt="attachment" />;
}
if (part.type === 'file' && part.mediaType === 'application/pdf') {
  return <iframe src={part.url} width={500} height={600} title="pdf" />;
}
```

Human-in-the-loop (Next.js)

ts

```
// Server — needsApproval: true
tools: {
  getWeatherInformation: tool({
    description: 'show the weather in a given city',
    inputSchema: z.object({ city: z.string() }),
    needsApproval: true,
    execute: async ({ city }) => 'sunny',
  }),
},
```

tsx

```
// Client — approval UI
import { lastAssistantMessageIsCompleteWithApprovalResponses } from 'ai';

const { messages, sendMessage, addToolApprovalResponse } = useChat({
  transport: new DefaultChatTransport({ api: '/api/chat' }),
  sendAutomaticallyWhen: lastAssistantMessageIsCompleteWithApprovalResponses,
});

// Render part.state === 'approval-requested', 'output-available', 'output-denied'
```

Dynamic approval: `needsApproval: async ({ amount }) => amount > 1000`.

Key version-change conclusions

v6 has consolidated three previously separate APIs into Core generation functions: structured output (`generateObject`/`streamObject` → `Output.*`), agent loops (manual stopping → `ToolLoopAgent`), and UI streaming (custom SSE → unified UI Message Stream protocol with typed `parts[]`). **Model references are now strings by default** via the AI Gateway, reducing boilerplate for provider swaps. **Tool definitions gained lifecycle hooks** (`onInputStart`, `onInputDelta`, `onInputAvailable`, `needsApproval`) that enable true streaming UIs and human-in-the-loop approval without custom plumbing. **MCP is production-ready** with HTTP/SSE/Streamable HTTP transports — stdio remains local-dev only. **Testing uses** `MockLanguageModelV3` (renamed from V2) with `LanguageModelV3Middleware` types. When building new applications in v6, prefer: string model IDs, `Output.*` for structure, `ToolLoopAgent` for orchestration, `DirectChatTransport` for in-process UIs, and `createUIMessageStream` with typed `data-*` parts for custom server→client channels.